



SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -2
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: TALARI VISHNUVARDHAN

Reg. No.: 192572091

Course Code: CSA0613

Slot: A

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

Course Faculty: Dr. Mary Harin Fernandez F

Q39. Combination Generation of Size K Using Backtracking

1. Title

Combination Generation of Size K Using Backtracking

2. Problem Statement

Given a set of elements and a value **K**, generate all possible combinations containing exactly **K elements**.

The algorithm must use **Backtracking** and select elements in increasing order to avoid duplicate combinations. It should also use **pruning** when the remaining elements are insufficient to complete a combination of size K.

3. Objective

The objectives are:

1. To generate all possible combinations of size K.
2. To use the Backtracking technique.
3. To avoid duplicate combinations.
4. To select elements in increasing order.
5. To apply pruning to reduce unnecessary computation.
6. To ensure all valid combinations are generated.

4. Input

The algorithm accepts:

- **Set of elements:** S
- **Number of elements:** n
- **Combination size:** K

Example

S = [1, 2, 3, 4]

K = 2

5. Output

All possible combinations of size K.

Example Output

[1, 2]

[1, 3]

[1, 4]

[2, 3]

[2, 4]

[3, 4]

6. Concept of Backtracking

Backtracking is an algorithmic technique that:

1. Selects an element.
2. Adds it to the current combination.
3. Recursively selects the next element.
4. Removes the previous element.
5. Tries another possible element.

This process continues until all valid combinations are generated.

7. Main Pseudocode

Combination Generation Using Backtracking

ALGORITHM GenerateCombinations(S, K)

INPUT:

S $\leftarrow$ Set of elements

K $\leftarrow$ Size of each combination

OUTPUT:

All possible combinations of size K

BEGIN

combination $\leftarrow$ Empty List

CALL Backtrack(0, combination)

END

8. Backtracking Pseudocode

PROCEDURE Backtrack(start, combination)

IF Size(combination) = K THEN

 PRINT combination

 RETURN

END IF

remaining $\leftarrow$ K - Size(combination)

IF Number of remaining elements < remaining THEN

 RETURN

END IF

FOR i $\leftarrow$ start TO Length(S) - remaining DO

 ADD S[i] TO combination

CALL Backtrack($i + 1$, combination)

REMOVE last element FROM combination

END FOR

END PROCEDURE

9. Complete Combined Pseudocode

This is the **best version to put in your report**:

ALGORITHM CombinationGeneration(S , K)

INPUT:

S = Set of elements

K = Required combination size

OUTPUT:

All combinations containing K elements

BEGIN

combination $\leftarrow$ Empty List

CALL Backtrack(0, combination)

END

PROCEDURE Backtrack(start, combination)

IF Size(combination) = K THEN

PRINT combination

RETURN

END IF

remaining $\leftarrow K$ - Size(combination)

IF Length(S) - start < remaining THEN

RETURN

END IF

FOR $i \leftarrow \text{start}$ TO $\text{Length}(S) - \text{remaining}$ DO

ADD $S[i]$ TO combination

CALL Backtrack($i + 1$, combination)

REMOVE last element FROM combination

END FOR

END PROCEDURE

10. Explanation of Inputs and Outputs

```
1 def combination(arr, k, start=0, result=[]):
2
3     if len(result) == k:
4         print(result)
5         return
6
7     for i in range(start, len(arr)):
8         result.append(arr[i])
9         combination(arr, k, i + 1, result)
10        result.pop()
11
12
13 arr = [1, 2, 3, 4]
14 k = 2
15
16 print("Combinations are:")
17 combination(arr, k)
```

Input

$S = [1, 2, 3, 4]$

$K = 2$

Where:

- S contains the available elements.
- K specifies the number of elements required in each combination.

Output

```
Combinations are:  
[1, 2]  
[1, 3]  
[1, 4]  
[2, 3]  
[2, 4]  
[3, 4]  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

[1, 2]

[1, 3]

[1, 4]

[2, 3]

[2, 4]

[3, 4]

11. Step-by-Step Working

Let:

$S = [1, 2, 3, 4]$

$K = 2$

Step 1

Start with an empty combination.

[]

Step 2

Select 1.

[1]

Step 3

Select 2.

[1, 2]

The combination size is equal to K.

Therefore:

PRINT [1, 2]

Step 4

Backtrack and replace 2 with 3.

[1, 3]

Step 5

Continue the process.

The algorithm generates:

[1, 2]

[1, 3]

[1, 4]

[2, 3]

[2, 4]

[3, 4]

12. Increasing Order Selection

The algorithm uses:

CALL Backtrack($i + 1$, combination)

This ensures that the next element is always selected after the current element.

For example:

[1, 2] ✓

[1, 3] ✓

[2, 3] ✓

But it does not generate:

[2, 1] ✗

[3, 1] ✗

[3, 2] ✗

Therefore, duplicate combinations are avoided.

13. Pruning

Pruning reduces unnecessary recursive calls.

The pruning condition is:

IF Length(S) - start < remaining THEN

 RETURN

END IF

This means:

If there are not enough remaining elements to complete a combination of size K, stop searching.

Example

Suppose:

S = [1, 2, 3, 4]

K = 3

If the algorithm is near the end of the set and only one element remains but two elements are required:

Remaining elements = 1

Required elements = 2

The algorithm stops immediately.

This saves computation time.

14. Algorithm Analysis

Let:

- n = Number of elements
- K = Size of combination

The number of combinations is:

$n!$

$K!(n-K)!$

This is written as:

$C(n, K)$

The algorithm must generate every valid combination.

Therefore, the time complexity depends on the number of combinations generated.

Time Complexity

$O(C(n, K) \times K)$

The factor K is required to create or print each combination.

Space Complexity

$O(K)$

The recursive combination stores a maximum of K elements.

15. Correctness of the Algorithm

The algorithm is correct because:

1. Every valid combination is considered

The algorithm recursively explores all possible selections.

2. Every combination contains exactly K elements

The algorithm prints a combination only when:

$\text{Size}(\text{combination}) = K$

3. Duplicate combinations are avoided

The algorithm always selects the next element using:

$i + 1$

Therefore, elements are selected in increasing order.

4. Invalid branches are removed

Pruning stops branches that cannot produce a valid combination.

Thus, the algorithm generates all valid combinations exactly once.

16. Industrial Applications

Combination generation using Backtracking is useful in many real-world problems.

1. Team Selection

Selecting K employees from n employees.

2. Resource Allocation

Selecting resources for a project.

3. Product Selection

Generating possible product bundles.

4. Network Design

Selecting possible network connections.

5. Testing

Generating combinations of software test cases.

6. Machine Learning

Feature selection from a large dataset.

17. Advantages

- Generates all valid combinations.
- Avoids duplicate results.
- Uses recursion efficiently.
- Pruning reduces unnecessary searching.
- Easy to understand and implement.
- Useful for optimization problems.

18. Limitations

- The number of combinations increases rapidly.
- Large input sets require more processing time.
- Backtracking may become expensive for very large values of n.

19. Result

For:

$S = [1, 2, 3, 4]$

$K = 2$

The algorithm successfully generates:

[1, 2]

[1, 3]

[1, 4]

[2, 3]

[2, 4]

[3, 4]

The total number of combinations is:

$C(4,2) = 6$

20. Conclusion

The Combination Generation problem was successfully solved using the **Backtracking technique**. The algorithm selects elements in increasing order, which prevents duplicate combinations. Recursion is used to build combinations step by step, while pruning eliminates branches that cannot produce a valid combination.

The algorithm guarantees that every valid combination of size K is generated exactly once. This approach is useful in industrial applications such as team selection, resource allocation, feature selection, software testing, and optimization problems.